



An introduction to Distributed Systems



Distributed Computing(COMP3008)

**Discipline of Computing School of Electrical Engineering, Computing
and Mathematical Sciences (EECMS)**

Copyright © 2018, Curtin University CRICOS Provide Code: 00301J

All Hail Mighty Windows

- By the way, you may want to get familiar with C#.
- We're using this because it is an excellent programming language for distributed systems.
- Microsoft has actually defined this market for two and a bit decades, so it's a solid learning ground.
- These principles can later be extended to other systems (sometimes quite easily)
- So.... Might want to get yourself some Windows. All Hail Mighty Windows

Java and C#

C# is commonly used for :

- developing Windows applications
- web applications using ASP.NET
- game development using Unity
- Backend services using the .NET framework. It is closely associated with Microsoft technologies and platforms.

Java is widely used for:

- developing cross-platform applications,
- including desktop applications,
- web applications (Java EE, Spring),
- mobile applications (Android), and enterprise-level systems.

Introduction to Visual Studio and C#

Visual Studio 2022

Open recent

As you use Visual Studio, any projects, folders, or files that you open will show up here for quick access.

You can pin anything that you open frequently so that it's always at the top of the list.

Get started



Clone a repository

Get code from an online repository like GitHub or Azure DevOps



Open a project or solution

Open a local Visual Studio project or .sln file



Open a local folder

Navigate and edit code within any folder



Create a new project

Choose a project template with code scaffolding to get started

[Continue without code →](#)

Create a new project

Recent project templates

 Console App (.NET Framework)

C#

Search for templates (Alt+S)



[Clear all](#)

C#

Windows

Console



Console App

A project for creating a command-line application that can run on .NET on Windows, Linux and macOS

C#

Linux

macOS

Windows

Console



Console App (.NET Framework)

A project for creating a command-line application

C#

Windows

Console

Not finding what you're looking for?
[Install more tools and features](#)

Configure your new project

Console App (.NET Framework)

C#

Windows

Console

Project name

ConsoleApp1

Location

C:\Users\290574c\source\repos

Solution name ⓘ

ConsoleApp1

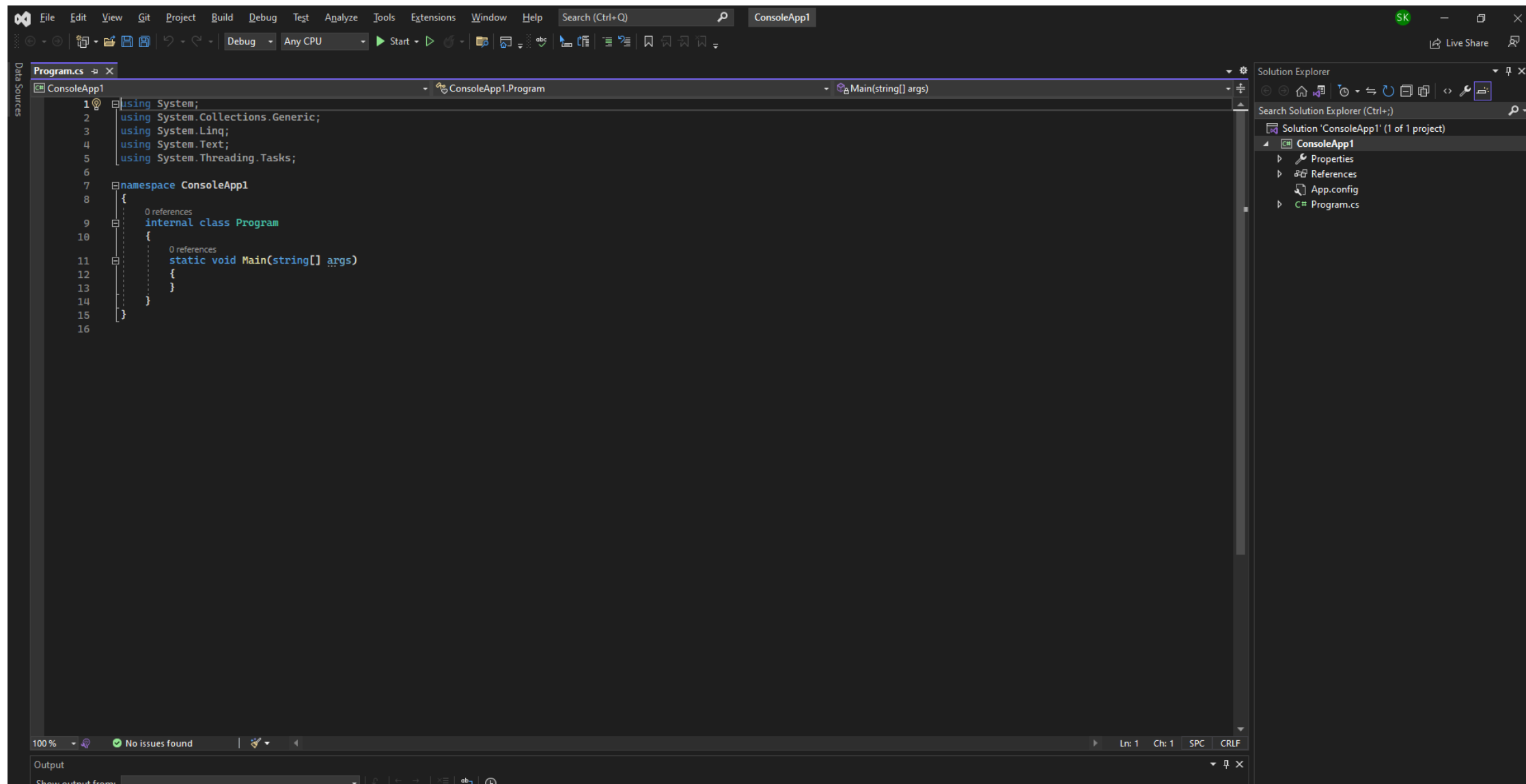
☐

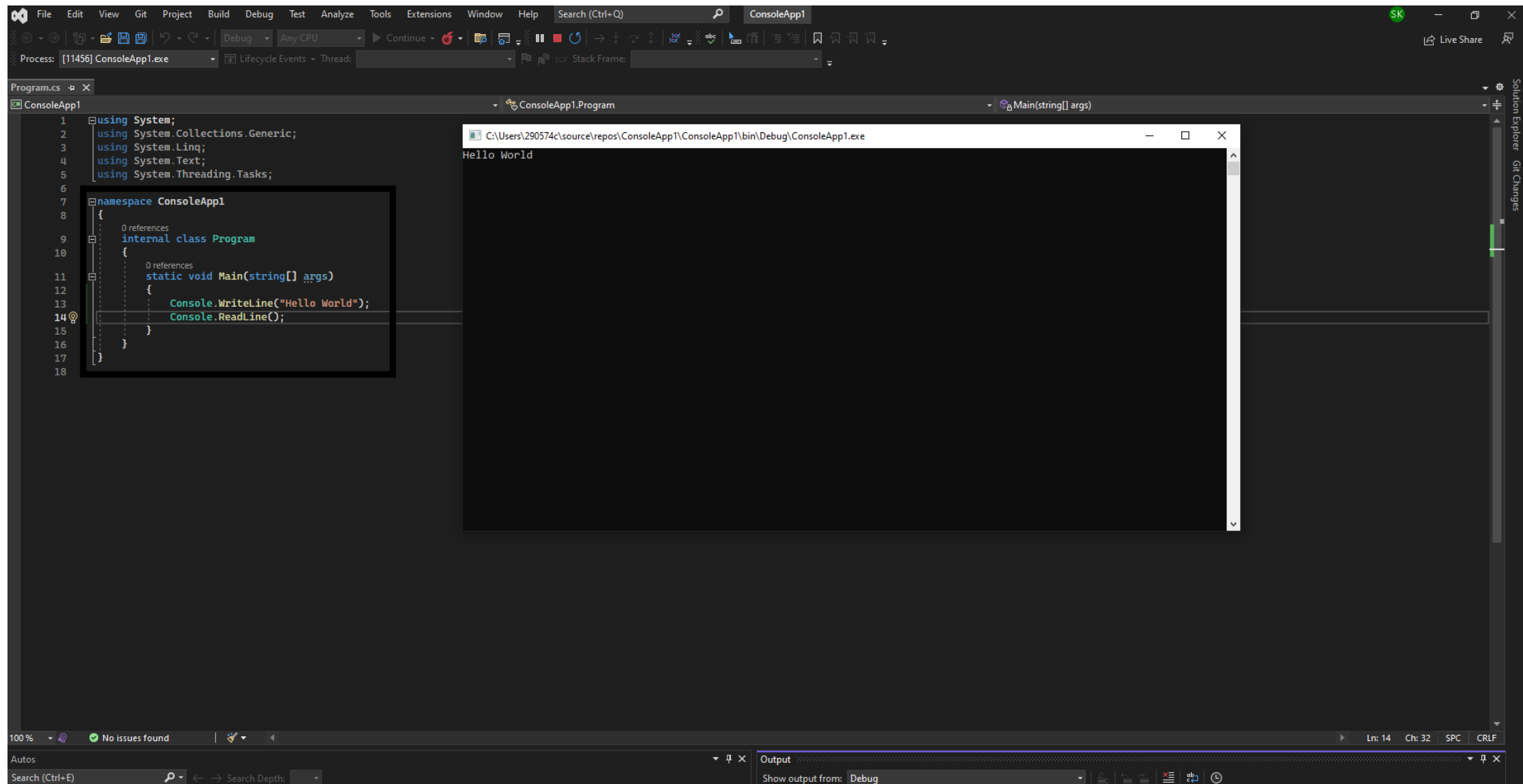
Place solution and project in the same directory

Framework

.NET Framework 4.7.2

Project will be created in "C:\Users\290574c\source\repos\ConsoleApp1\ConsoleApp1\."





Visual Studio interface showing a C# console application named ConsoleApp2. The main code file, Program.cs, contains the following code:

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace ConsoleApp2
8 {
9     0 references
10     internal class Program
11     {
12         0 references
13         static void Main(string[] args)
14         {
15             Console.WriteLine("Hello word");
16             Console.ReadLine();
17         }
18 }
```

The code is annotated with two errors in the Error List:

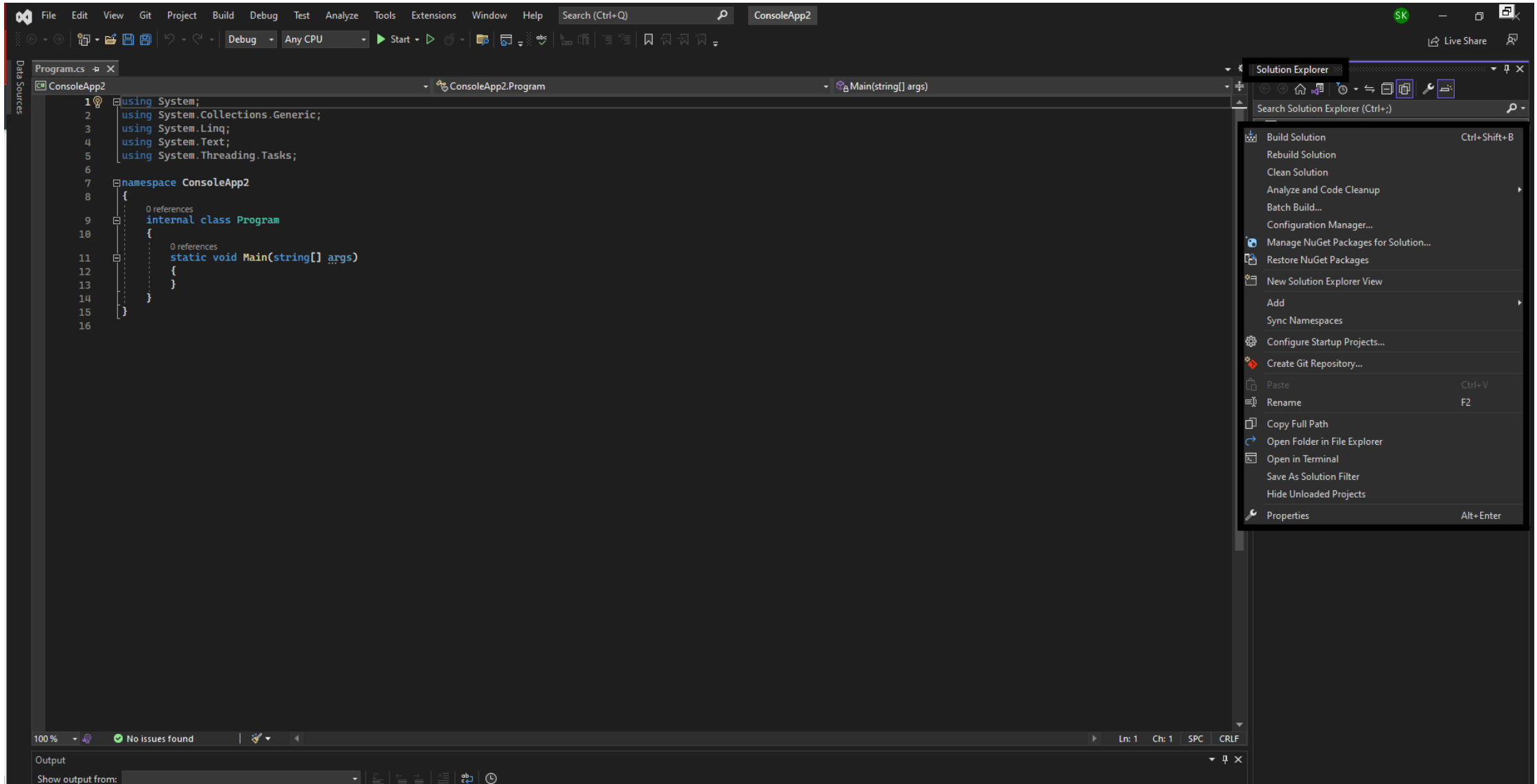
- CS1519 Invalid token ')' in class, record, struct, or interface member declaration
- CS1002 ; expected

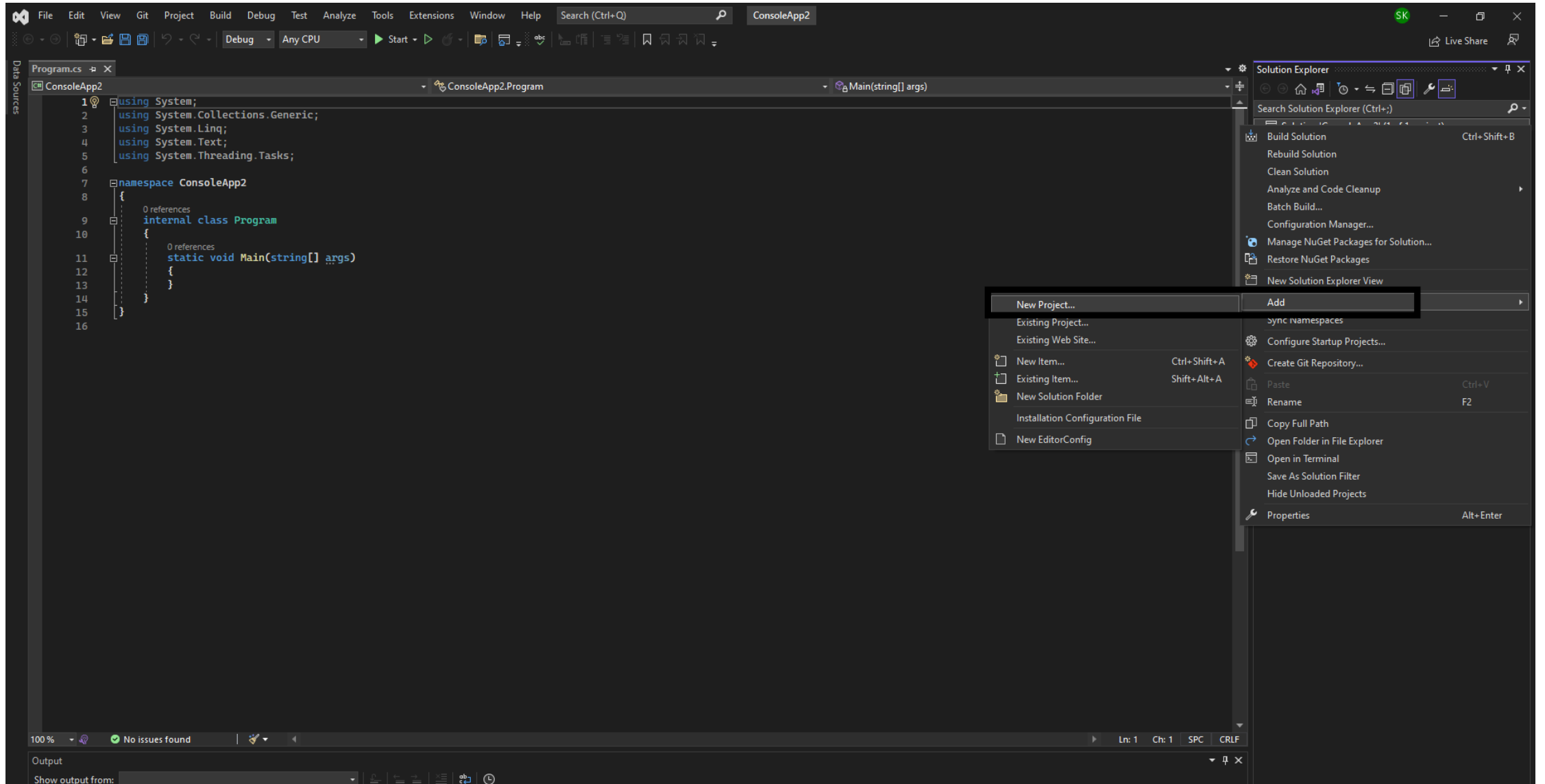
The Solution Explorer on the right shows the project structure:

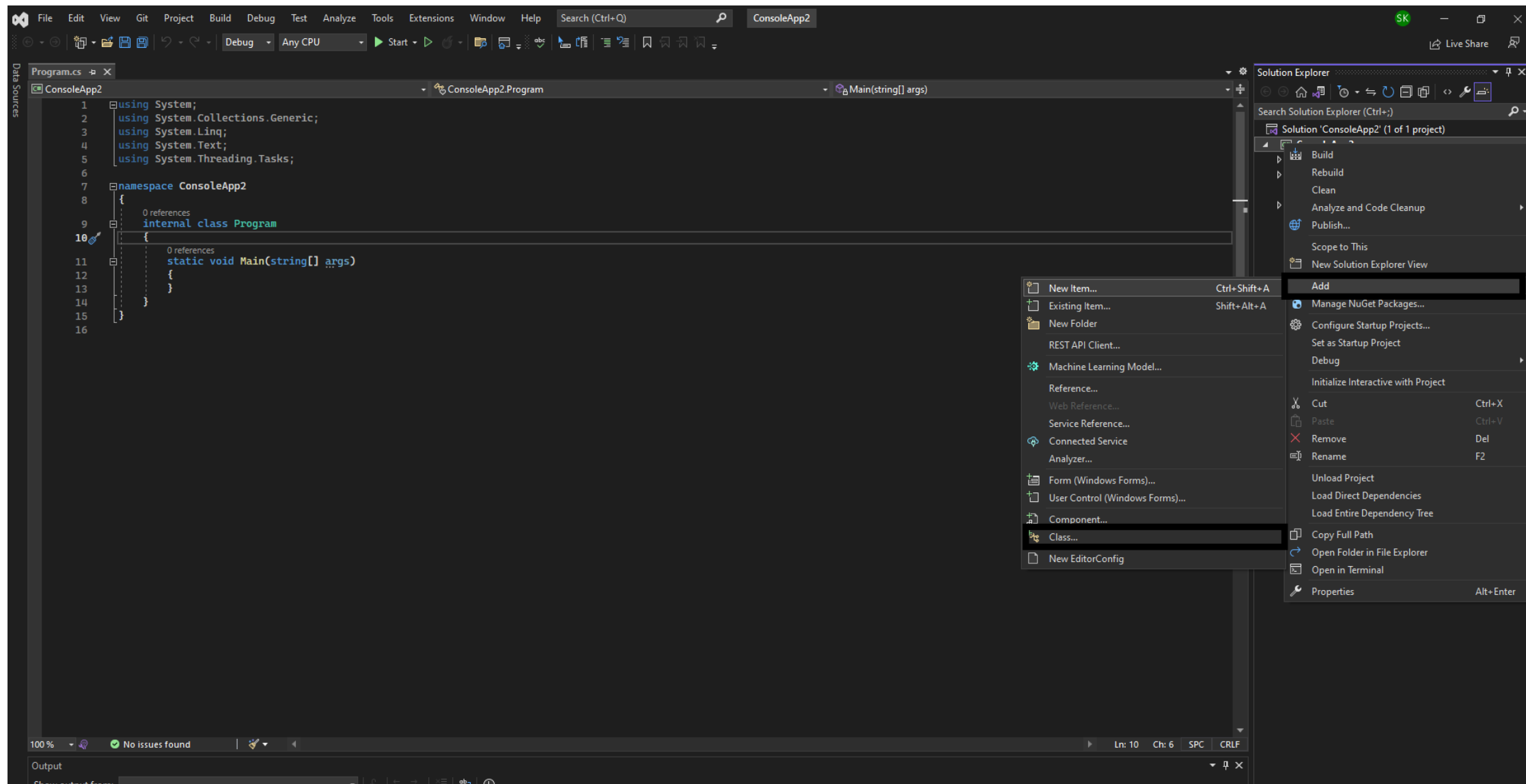
- ConsoleApp2
 - Properties
 - References
 - App.config
 - Class1.cs
 - FileName.cs
 - Program.cs

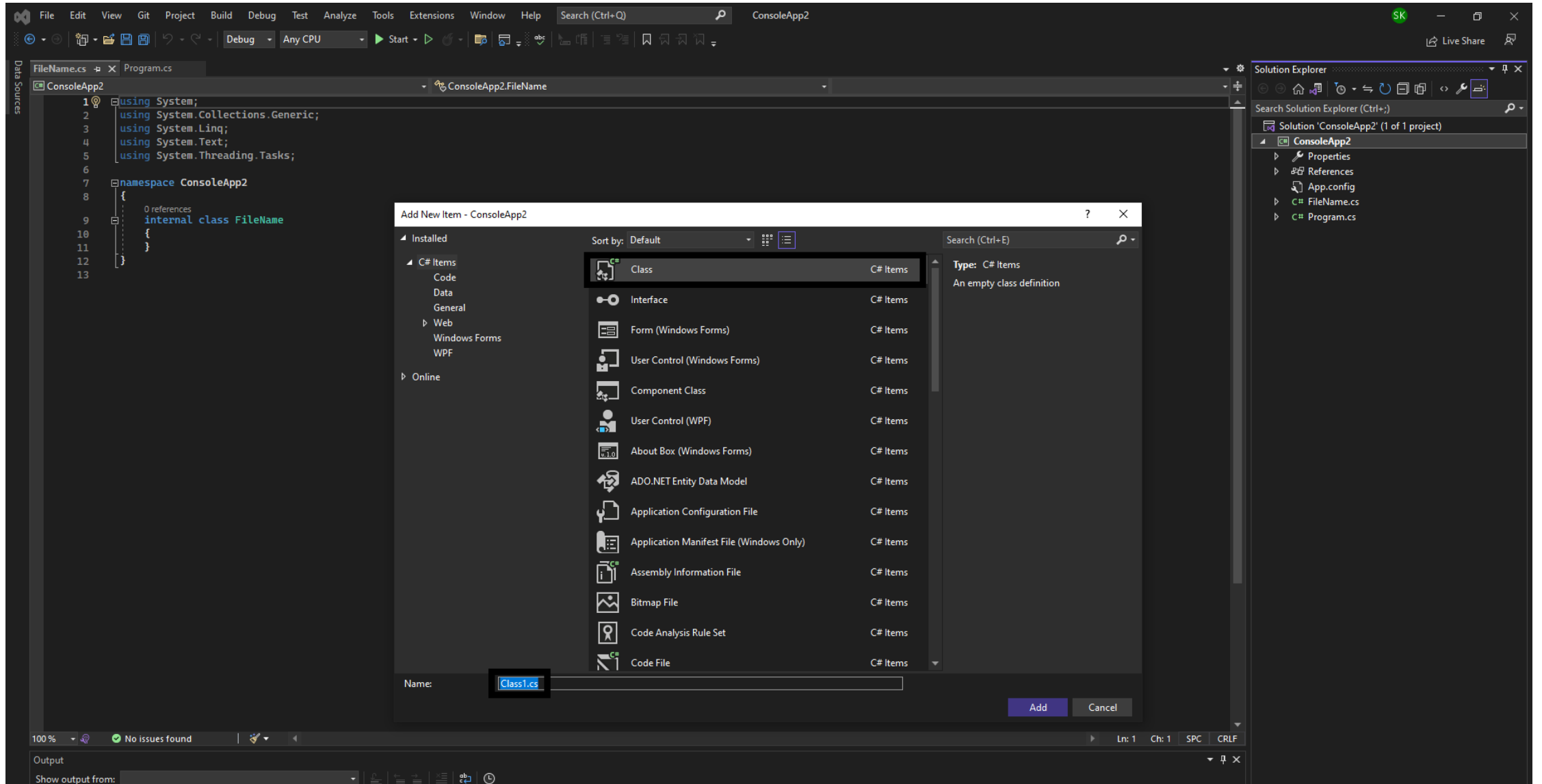
The Error List pane at the bottom displays the following table:

Project	File	Line	Suppression State
ConsoleApp2	Class1.cs	12	Active
ConsoleApp2	Program.cs	13	Active



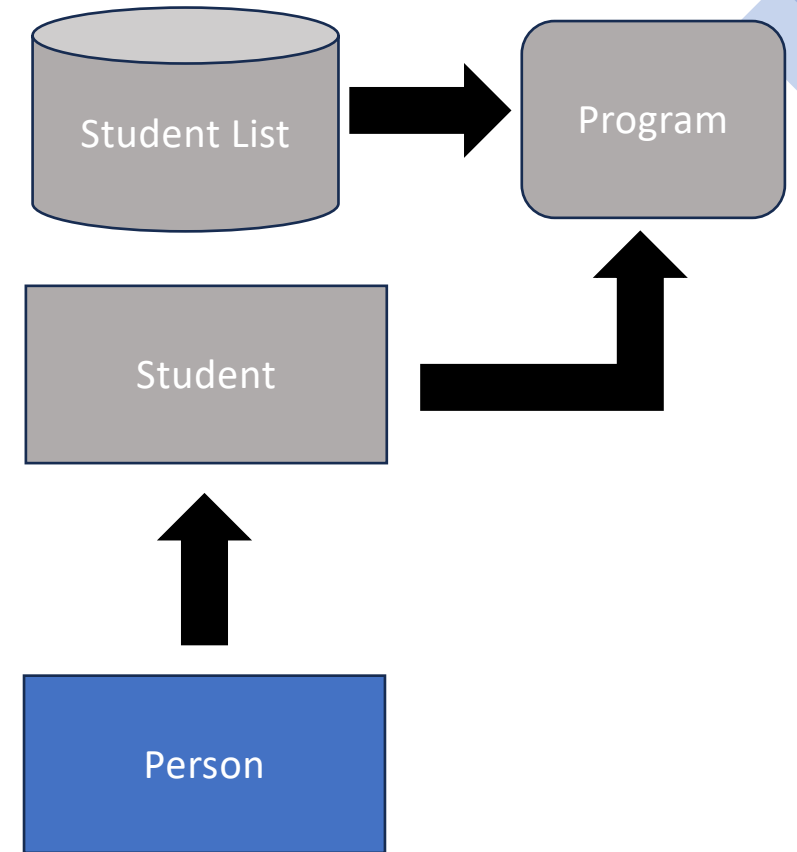






Person

```
7 namespace ConsoleApp1
8 {
9     1 reference
10     internal class Person
11     {
12         private string name;
13         4 references
14         public string Name {
15             get { return name; }
16             set { name = value; }
17         }
18     }
```



Access Modifier

- Access modifiers in C# are keywords used to specify the accessibility or visibility of classes, members (methods, properties, fields, etc.), and types within a program. They determine which parts of a program can access or modify a particular class or member.
- `public int publicField; // Accessible from anywhere`
- `private int privateField; // Accessible within the same class`
- `protected int protectedField; // Accessible within the same class and derived classes`
- `internal int internalField; // Accessible within the same assembly`
- `protected internal int protIntField; // Accessible within the same assembly or derived classes`
- `private protected int privProtField; // Accessible within the same assembly and derived classes`

Properties

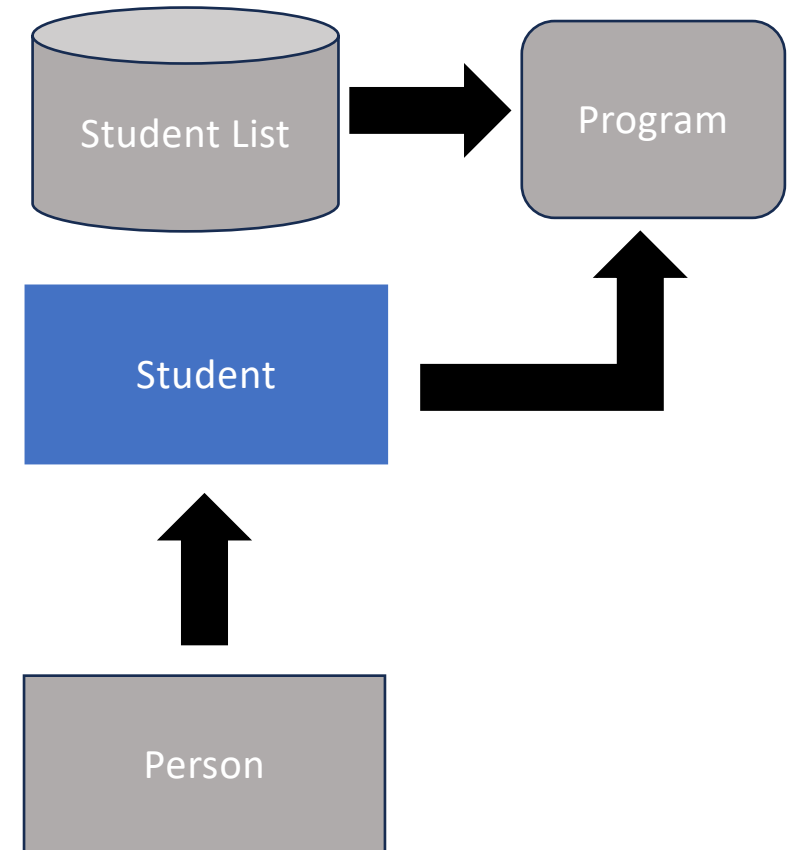
- C#, a property is a class member that encapsulates access to a private field or provides a computed value. It provides a way to define getter and setter methods to control how the field's value is accessed and modified.

```
public class Person
{
    private string name;
    public string Name {
        get { return name; }
        set { name = value; }
    }
}
```



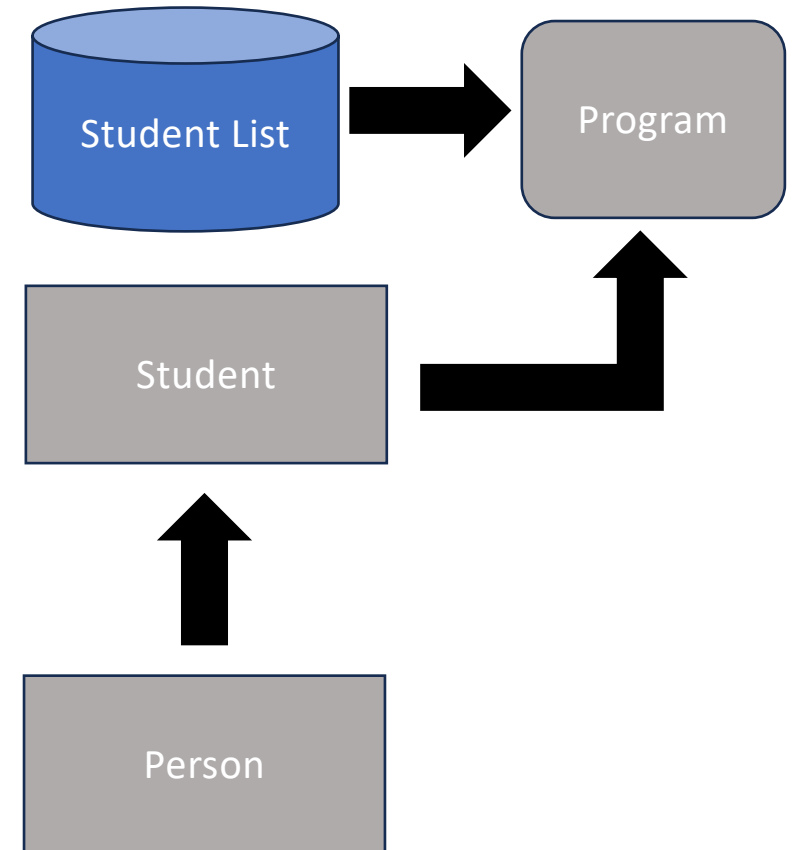
Student

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace ConsoleApp1
8 {
9     11 references
10     internal class Student: Person
11     {
12         private int id;
13         private string university;
14
15         3 references
16         public int Id
17         {
18             get { return id; }
19             set { id = value; }
20         }
21
22         3 references
23         public string University
24         {
25             get { return university; }
26             set { university = value; }
27         }
28
29         0 references
30         public override string ToString()
31         {
32             string info = "The student's name is " + Name + "\n";
33             info=info + "The student's id is " + id + "\n";
34             info = info+ "The student's university is " + university;
35             return info;
36         }
37     }
38 }
```

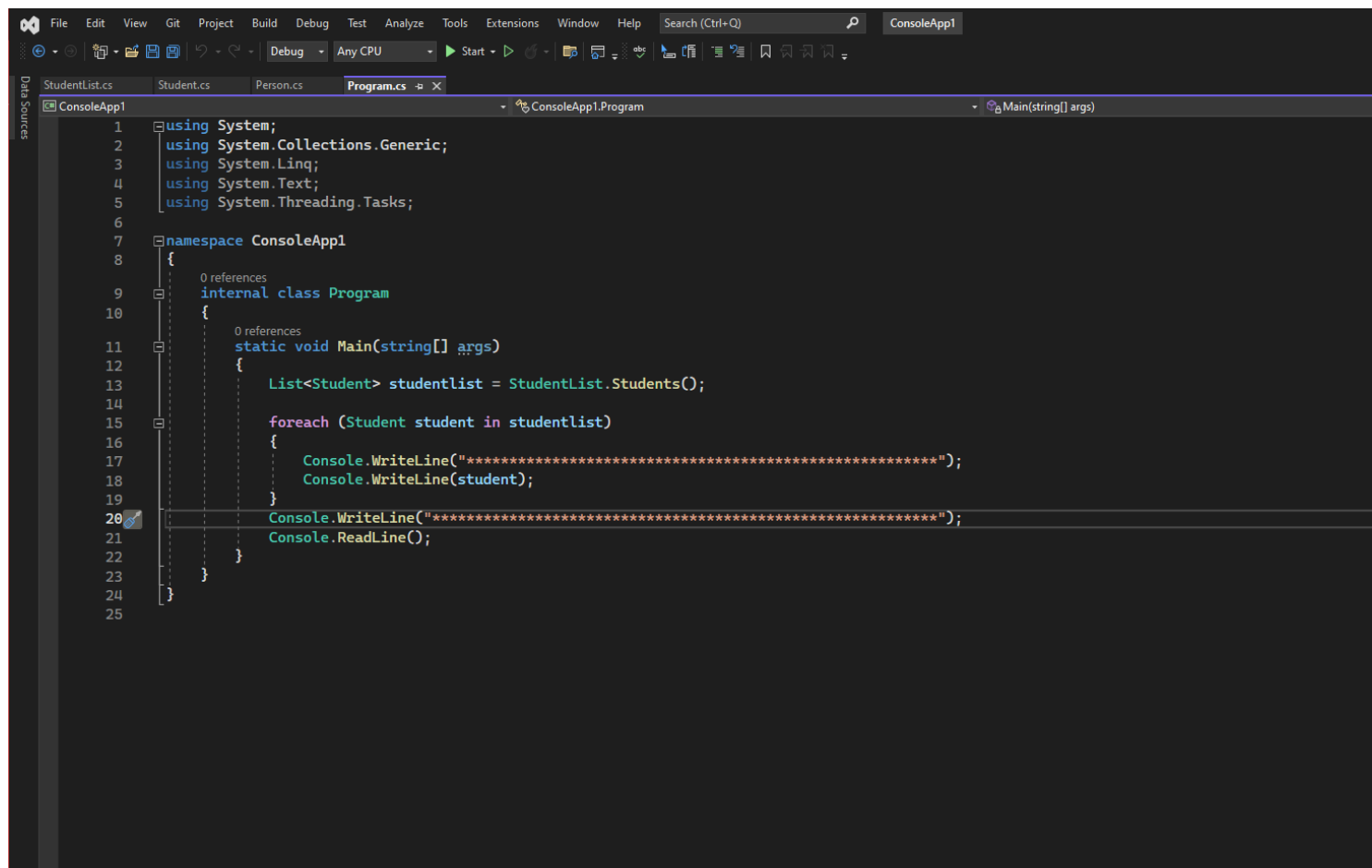


Student List

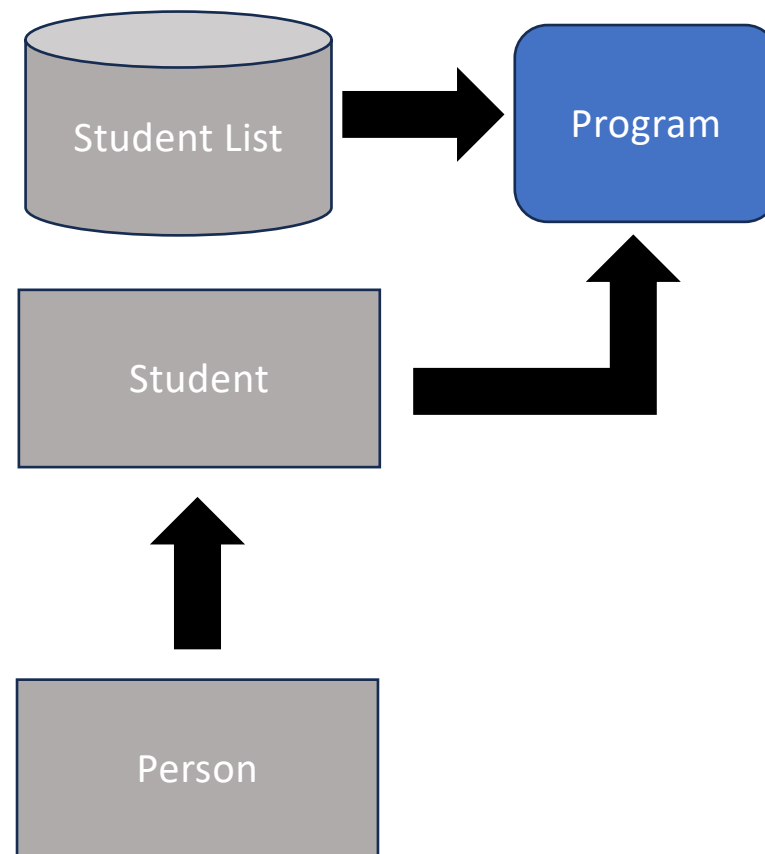
```
File Edit View Git Project Build Debug Test Analyze Tools Extensions Window Help Search (Ctrl+Q) ConsoleApp1
Debug Any CPU Start
Studentlist.cs Student.cs Person.cs Program.cs ConsoleApp1.StudentList Students()
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace ConsoleApp1
8 {
9     1 reference
10     internal class StudentList
11     {
12         1 reference
13         public static List<Student> Students()
14         {
15             List<Student> slist = new List<Student>();
16
17             Student student1 = new Student();
18             student1.Name = "Robert";
19             student1.University = "Curtin";
20             student1.Id = 101;
21
22             Student student2 = new Student();
23             student2.Name = "Mia";
24             student2.University = "EWU";
25             student2.Id = 102;
26
27             Student student3 = new Student();
28             student3.Name = "Adam";
29             student3.University = "Murdoch";
30             student3.Id = 103;
31
32             slist.Add(student1);
33             slist.Add(student2);
34             slist.Add(student3);
35
36             return slist;
37         }
38     }
39 }
```



Program



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace ConsoleApp1
8 {
9     0 references
10     internal class Program
11     {
12         0 references
13         static void Main(string[] args)
14         {
15             List<Student> studentlist = StudentList.Students();
16
17             foreach (Student student in studentlist)
18             {
19                 Console.WriteLine("*****");
20                 Console.WriteLine(student);
21             }
22             Console.WriteLine("*****");
23             Console.ReadLine();
24         }
25     }
```



Visual Studio interface showing a C# console application named ConsoleApp1. The code in Program.cs defines a namespace ConsoleApp1 with an internal class Program containing a static void Main method. The Main method uses StudentList.Students() to get a list of students and iterates through them, printing their details.

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace ConsoleApp1
8 {
9     0 references
10     internal class Program
11     {
12         0 references
13         static void Main(string[] args)
14         {
15             List<Student> studentlist = StudentList.Students();
16
17             foreach (Student student in studentlist)
18             {
19                 Console.WriteLine("*****");
20                 Console.WriteLine(student);
21             }
22             Console.WriteLine("*****");
23             Console.ReadLine();
24         }
25     }
```

The output window displays the results of the program execution, showing student details for Robert, Mia, Adam, and Murdoch.

```
C:\Users\290574c\Downloads\ConsoleApp1 (1)\ConsoleApp1\ConsoleApp1\bin\Debug\ConsoleApp1.exe
*****
The student's name is Robert
The student's id is 101
The student's university is Curtin
*****
The student's name is Mia
The student's id is 102
The student's university is Ewu
*****
The student's name is Adam
The student's id is 103
The student's university is Murdoch
*****
```

The bottom status bar shows the Autos window with a search bar and the Output window displaying the program's output. A notification for "Input language switching" is also visible.

Study Resources

- <https://www.w3schools.com/cs/index.php>
- <https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/tutorials/>
- <https://dotnet.microsoft.com/en-us/learn>

What even is a distributed computing anyway?

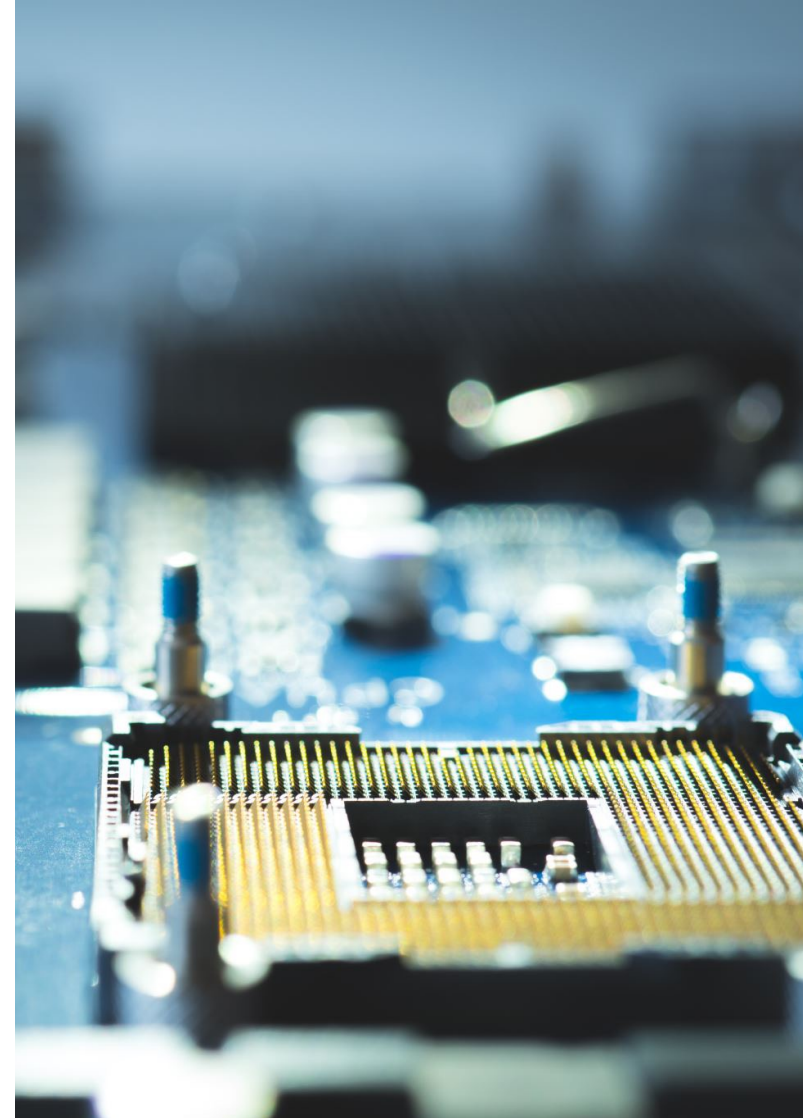
- Processors are fast! (but can only go so fast)
- More processors == More fast!
- Concurrent computing == Great! (ignoring the headache of synchronising stuff)
- Problem: I only have 1 processor (in 1990's anyway)
- Solution: Use other PC's processors!

And why would anyone care?

- Distributed number crunching is great and all, but you know what else is cool? The Internet.
- People all have computers, but those are far away from my database/application.
- Why not distribute parts of my application to those computers?
- And this is basically how everything works now

To Summarize

- Distributed computing is splitting applications into multiple processes on multiple machines.
- Requires a lot of inter-process communication (IPC)
- Is not a simple Client-Server architecture!
 - The application must be doing some useful work in multiple places.
 - However that's pretty much how all Client-Server apps work these days anyway, so it is confusing.



Client, Server, Peer to Peer

- Client-Server application:
 - Dumb client accesses smart server, does all work up there.
 - Example: SSH, FTP, plain HTTP.
- Peer to Peer application:
 - No servers, clients all talk amongst each other, however no coordinated, orchestrated job is being done.
 - Example: BitTorrent, Mesh Wi-Fi networks.
- Distributed Application:
 - Can be Peer to Peer (Cryptocurrencies), can be Client-Server (MMORPGs).
 - Coordinated, singular application being run across all member computers.

Are Browsers Distributed?

- Browsers themselves? No. browser itself is a local application in our computers but the applications that we access through the browser might be distributed
 - Basic HTML pages aren't either.
- What runs on them is these days.
 - Online Documents
 - Webmail Clients
 - Online Games
 - Web Crypto Miners (although you may wish they weren't)



So you want to distribute your app

- You'll need an IPC method.
 - Needs to support networks
 - Although quite a few of these predate modern networking!
 - You might just think to use TCP/UDP sockets.
 - This is a less than great idea.
-

But I like the TCP/IP stack!

- TCP/IP is too general purpose
 - Sure you can send packets, but every application needs you to re-invent the wheel and build a protocol on top of TCP/IP!
 - What you want is a protocol that hides all the messiness of networking
 - Ideally so the programmer can't see the distribution.
 - Pro: You don't need special skills/design principles to be a distributed programmer
-

Remote Procedure Calls

- Programs are made of function calls.
- So.... Why not have a function call that actually happens somewhere else?
 - Maybe in another process, maybe on another computer
 - It should look the same as a regular one
- Very old concept, original RFC 707 was in 1976!
 - Xerox tried it first (because of course they did)
 - Sun's ONC RPC system was the first popular one.
 - Many other systems exist.
 - Most people just use the web these days.

RPC - understanding the Basics

- In a traditional local procedure call, you invoke a function directly in your code, and it runs in the same address space as your program.
- In an RPC, the function is executed on a remote server, and you only send a request to the server to execute the function on your behalf.
- The server processes the request and sends back the result to the client.



Choosing an RPC Framework

- There are several RPC frameworks available, each with its own pros and cons. Some popular ones include:
 - gRPC (Google RPC)
 - Apache Thrift
 - JSON-RPC
 - XML-RPC
- 

RPC-Defining the Interface

- Before you can use RPC, you need to define the interface of the functions you want to call remotely. This typically involves creating an IDL (Interface Definition Language) file where you specify the functions, their parameters, and return types.
- The IDL file is then used to generate code for both the client and the server.
- Using this IDL file, you can generate client and server code in various programming languages, such as C++, Java, Python, etc. The client code will contain stub functions to make remote calls, and the server code will contain the actual implementation of the functions defined in the IDL. This way, you can achieve remote procedure calls between client and server components seamlessly.

```
idl Copy code

// Example Calculator IDL

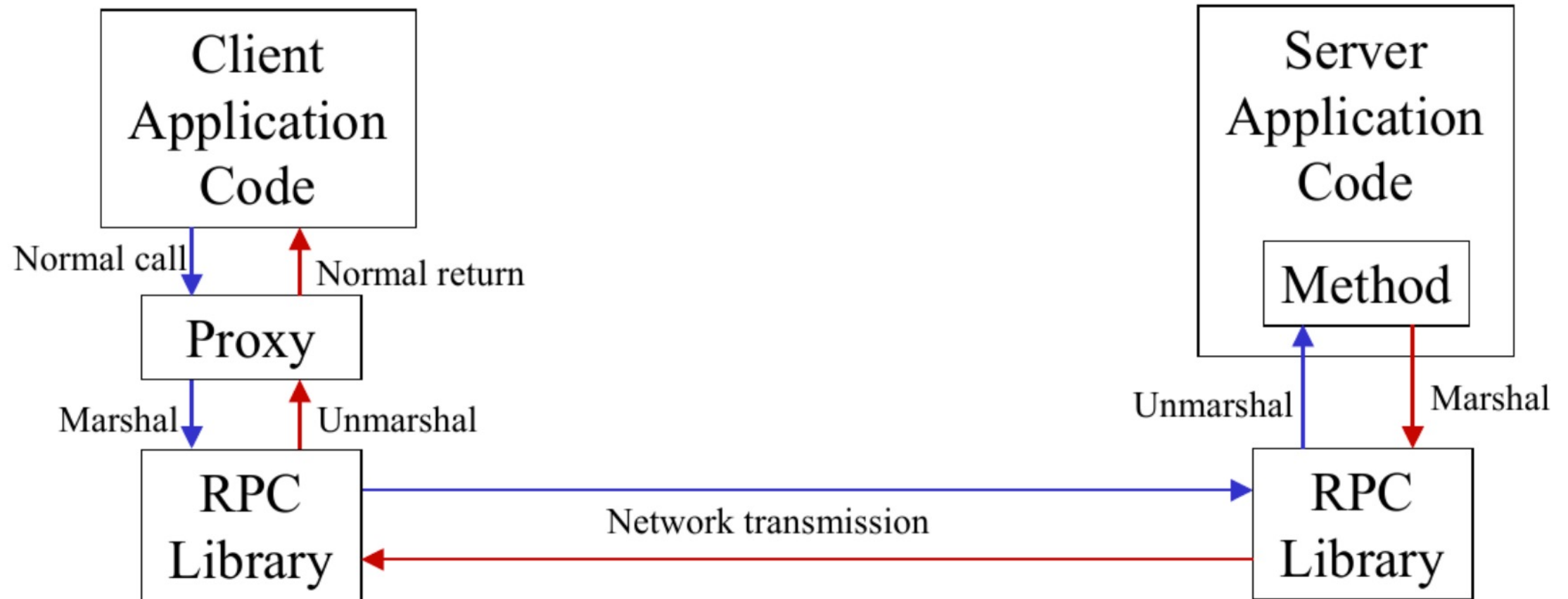
module Calculator {

    // Function to add two numbers
    int add(in int num1, in int num2, out int result);

    // Function to subtract two numbers
    int subtract(in int num1, in int num2, out int result);

};
```

RPC – Architecture



RPC – Implementing the Server

- On the server side, you need to implement the functions defined in the IDL file.
- The server listens for incoming RPC requests from clients.
- When a request comes in, the server executes the corresponding function and returns the result.



RPC – Implementing the client



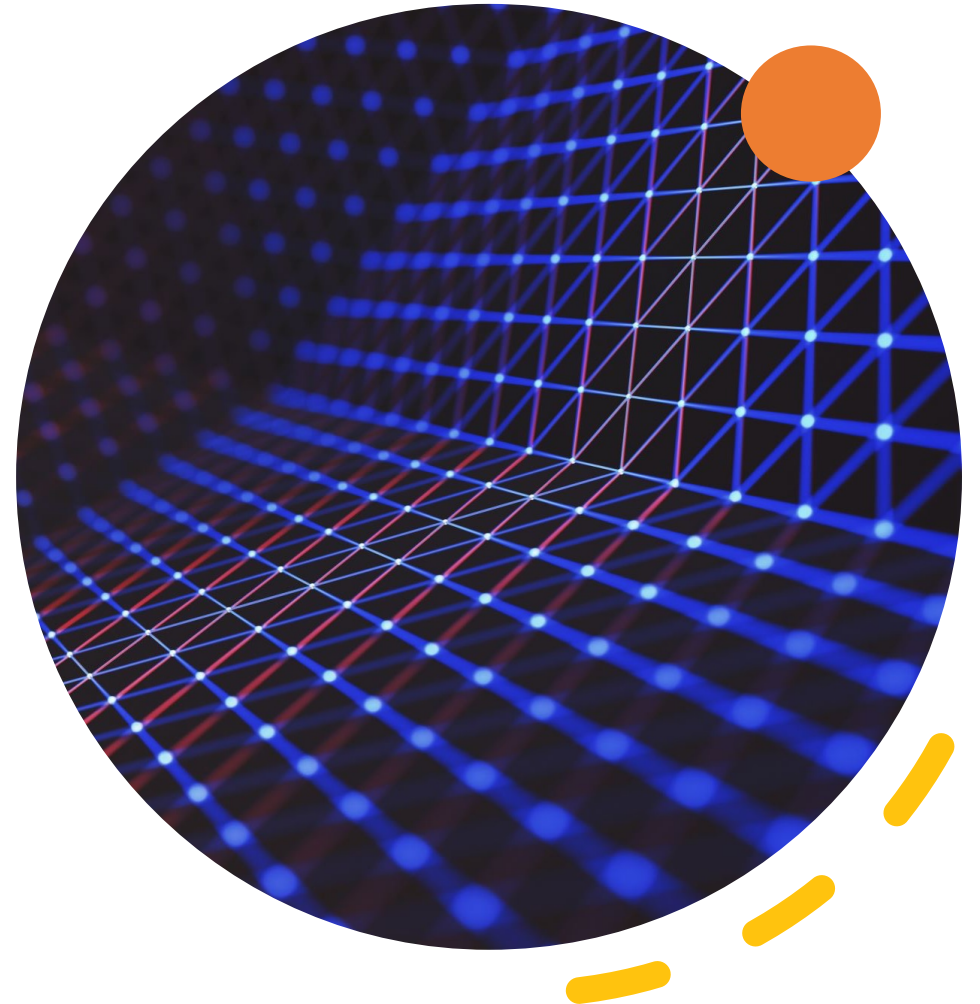
On the client-side, a client library is generated from the IDL file, which provides stub functions corresponding to the server's functions.



The client uses these stub functions to make RPC requests to the server.



The client then waits for the response from the server and processes the result.



RPC - **Handling Errors**

In an RPC system, various issues can occur, such as network failures or the server being unavailable.

Both the client and server should handle errors gracefully to ensure reliability and robustness.





Serialization and Deserialization

- RPC involves sending data across the network. Before sending data, it needs to be converted into a format suitable for transmission (serialization).
 - Serialization involves converting complex data structures (e.g., objects, classes) into a sequence of bytes or text that can be easily reconstructed into the original data structure when needed.
 - Common serialization formats include JSON, XML, Protocol Buffers, and MessagePack, among others.
 - Once the data reaches its destination, it is converted back to its original format (deserialization).
- 



Marshaling and Serialization

- Marshaling deals with converting data between different representations within the same environment (e.g., within the same operating system)
- Serialization deals with converting data into a portable format for transfer or persistence, often between different environments or systems.
- Both marshaling and serialization play essential roles in distributed computing and data exchange scenarios.

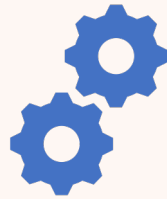
RPC - Considerations



Security Considerations:

RPC involves communication over a network, and security is a crucial aspect.

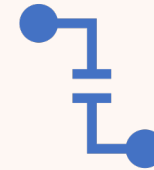
Ensure that you use secure channels (e.g., HTTPS) to protect data during transmission and consider authentication and authorization mechanisms.



Testing and Debugging:

Test your RPC implementation thoroughly to ensure it works as expected.

Use debugging tools to identify and fix any issues that may arise.



Scaling:

RPC can be a powerful tool for distributed systems, but as the system grows, you may need to consider load balancing and other scaling strategies to handle increased traffic and demand.



RPC Pros and Cons

➤ Pros:

- It's just like calling a local function
- Messaging requires you building a lot of network code, not so with RPC systems

➤ Cons:

- Remote calls have weird problems:
- Much slower due to networks
- Can fail due to networks
- You have no idea what happened or where you got up to... due to networks

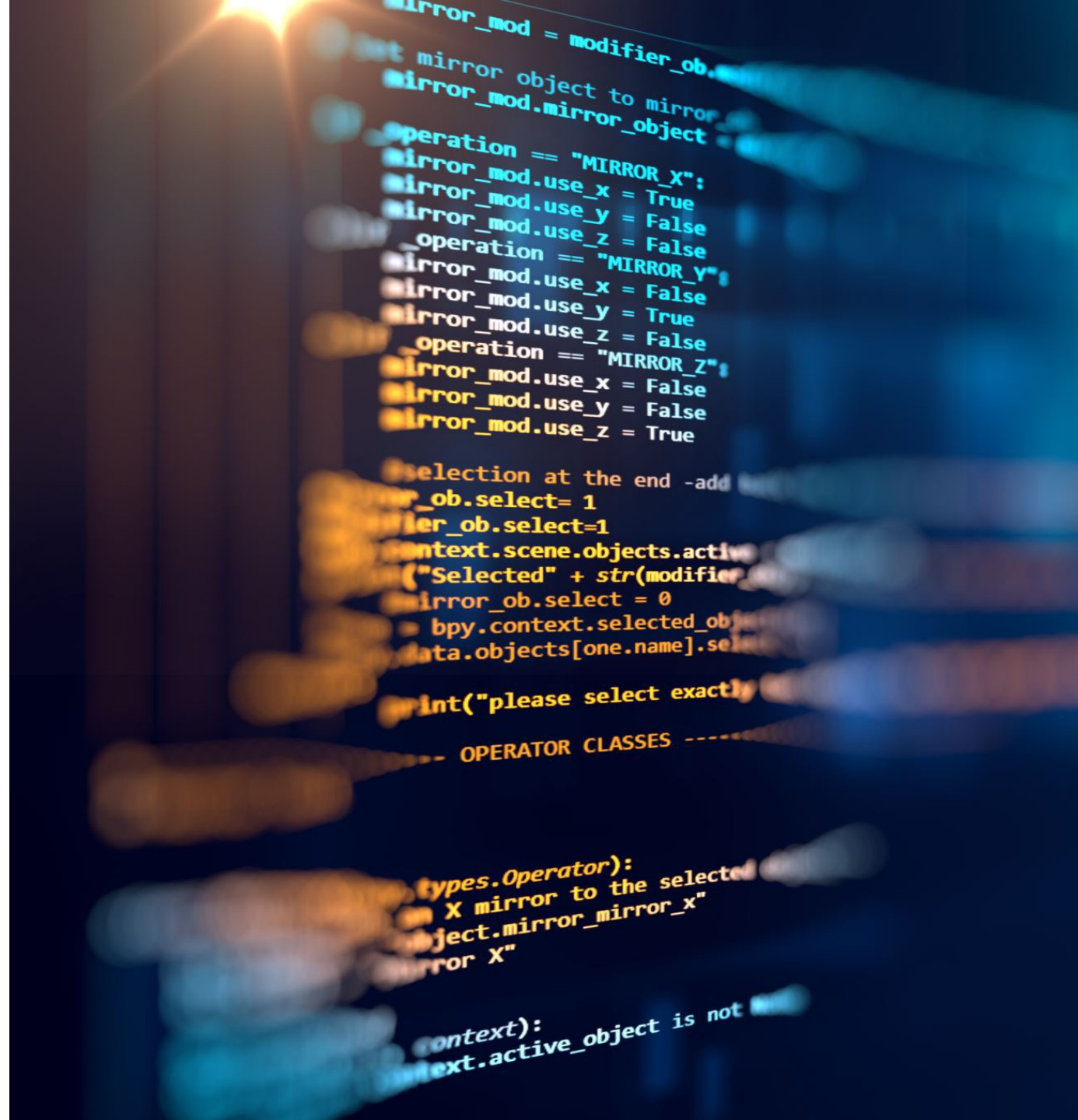


RPC Cons continued

- Worst of all, the system looks just like local function calls.
 - This means programmers can have a hard time understanding that things are distributed.
 - This leads to incomprehensible bugs caused by networking errors (how can you reference memory over a network?).
 - This is crippling for projects that are highly distributed.

Working with .NET Windows Presentation Foundation (WPF)

- <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/?view=netdesktop-7.0>
- WPF provides a comprehensive set of application-development features that include Extensible Application Markup Language (XAML).
- WPF is part of .NET, so you can build applications incorporating other elements of the .NET API.



Visual Studio IDE showing a C# console application named ConsoleApp1. The code defines a `Person` class with `Id` and `University` properties, and a `ToString()` method.

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace ConsoleApp1
8 {
9     11 references
10     internal class Student: Person
11     {
12         private int id;
13         private string university;
14
15         3 references
16         public int Id
17         {
18             get { return id; }
19             set { id = value; }
20         }
21
22         3 references
23         public string University
24         {
25             get { return university; }
26             set { university = value; }
27         }
28
29         0 references
30         public override string ToString()
31         {
32             string info = "The student's name is " + Name + "\n";
33             info=info + "The student's id is " + id + "\n";
34             info = info+ "The student's university is " + university;
35             return info;
36         }
37     }
38 }
```

The Solution Explorer shows the project structure, including `Person.cs` and `StudentList.cs`. The Output window is empty.

Context menus are visible over the code editor:

- Left Context Menu (over `Person`):**
 - New Project...
 - Existing Project...
 - Existing Web Site...
 - New Item... (Ctrl+Shift+A)
 - Existing Item... (Shift+Alt+A)
 - New Solution Folder
 - Installation Configuration File
 - New EditorConfig
- Right Context Menu (over `ToString()`):**
 - Build Solution (Ctrl+Shift+B)
 - Rebuild Solution
 - Clean Solution
 - Analyze and Code Cleanup
 - Batch Build...
 - Configuration Manager...
 - Manage NuGet Packages for Solution...
 - Restore NuGet Packages
 - New Solution Explorer View
 - Add
 - Sync Namespaces
 - Configure Startup Projects...
 - Create Git Repository...
 - Paste (Ctrl+V)
 - Rename (F2)
 - Copy Full Path
 - Open Folder in File Explorer
 - Open in Terminal
 - Save As Solution Filter
 - Hide Unloaded Projects
 - Properties (Alt+Enter)

Add a new project

Recent project templates

A list of your recently accessed templates will be displayed here.

WPF

×

▼

[Clear all](#)

All languages ▼ All platforms ▼ All project types ▼

 **WPF Application**
A project for creating a .NET WPF Application
C# Windows Desktop

 **WPF Application**
A project for creating a .NET WPF Application
Visual Basic Windows Desktop

 **WPF Class Library**
A project for creating a class library that targets a .NET WPF Application
C# Windows Desktop Library

 **WPF Class Library**
A project for creating a class library that targets a .NET WPF Application
Visual Basic Windows Desktop Library

 **WPF Custom Control Library**
A project for creating a custom control library for .NET WPF Applications
C# Windows Desktop Library

 **WPF Custom Control Library**
A project for creating a custom control library for .NET WPF Applications
Visual Basic Windows Desktop Library

Next

Configure your new project

WPF Application

C#

Windows

Desktop

Project name

WpfApp1

Location

C:\Users\283602K\Downloads\DC Materials\Lecture-1\DC_Lecture1_Demo\ConsoleApp1

...

Project will be created in "C:\Users\283602K\Downloads\DC Materials\Lecture-1\DC_Lecture1_Demo\ConsoleApp1\WpfApp1\"

Back

Next

Additional information

WPF Application

C#

Windows

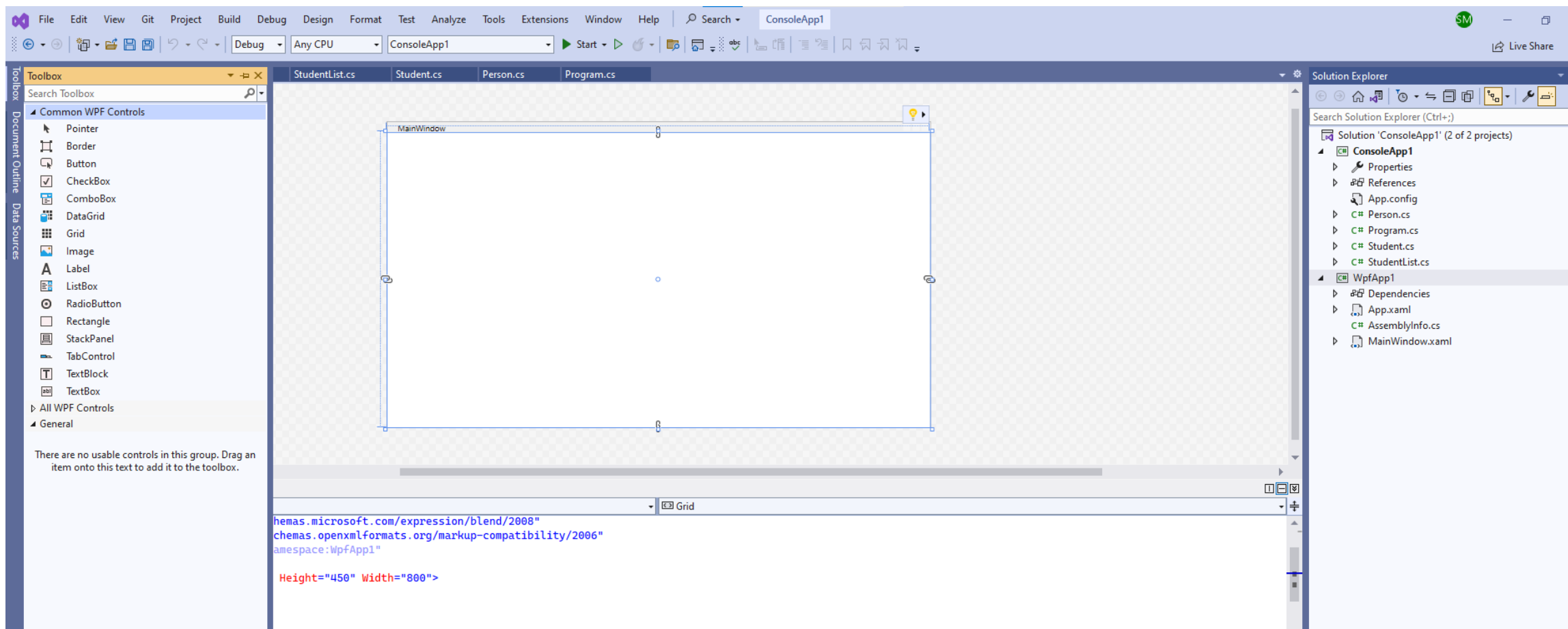
Desktop

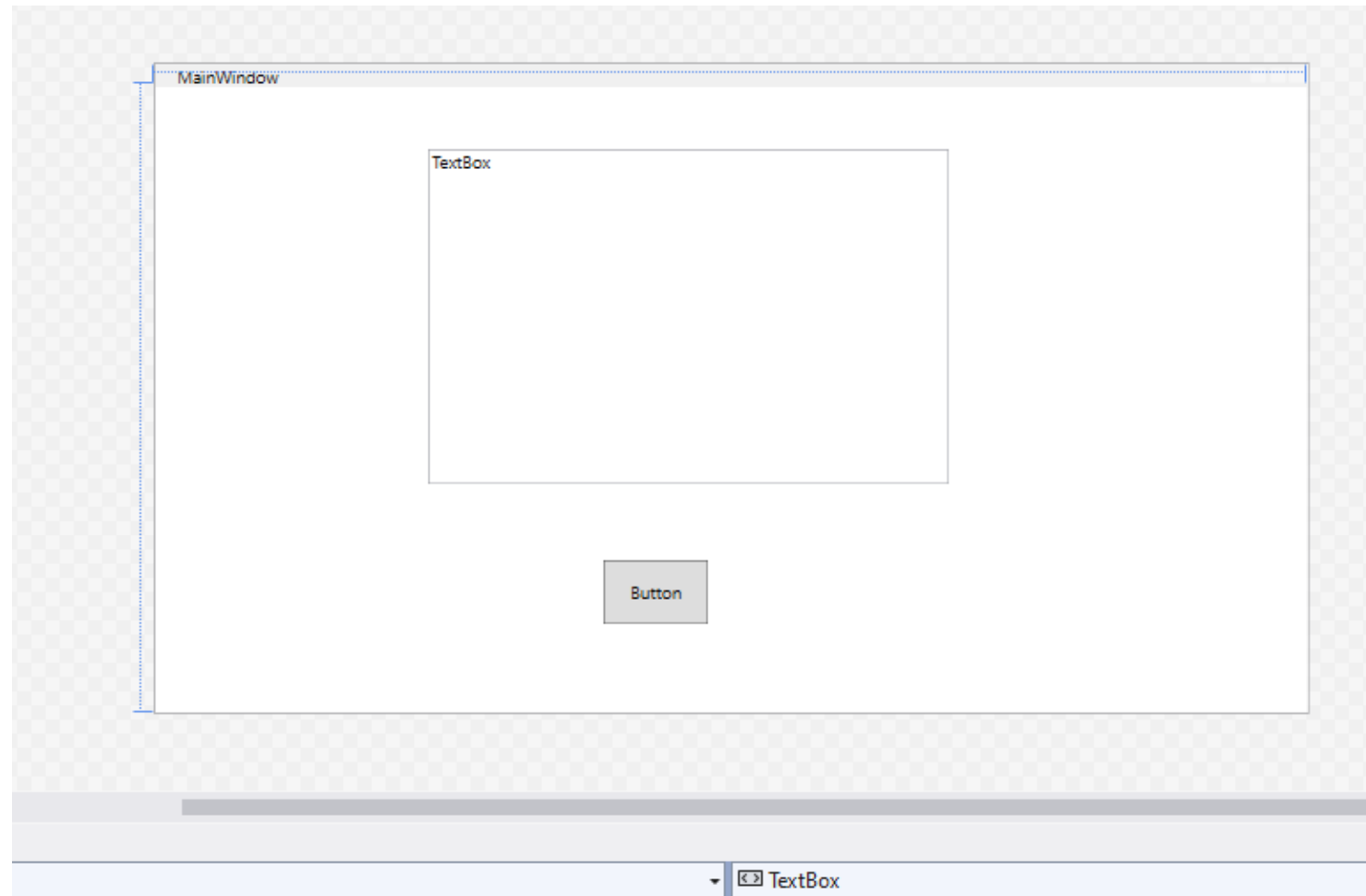
Framework 

.NET 6.0 (Long Term Support) 

Back

Create





openxmlformats.org/markup-compatibility/2006"

xmlns:WpfApp1"

Height="450" Width="800">

Content="Left" Margin="189,43,0,0" TextWrapping="Wrap" Text="TextBox" VerticalAlignment="Top" Width="362"

MainWindow

328

311

GET ALL

pace:WpfApp1"

ght="450" Width="800">

gnment="Left" Margin="189,43,0,0" TextWrapping="Wrap" VerticalAlignment="Top" Width="362" Height="232"/>

LL" HorizontalAlignment="Left" Margin="311,328,0,0" VerticalAlignment="Top" Height="45" Width="73" RenderTransformOrigin="1.14,0.449" Click="Button_Click"/>

Ln: 11 Ch: 16 SPC CRLF

Search Solution Explorer (Ctrl+)

Solution 'ConsoleApp1' (2 of 2 projects)

- ConsoleApp1
 - Properties
 - References
 - App.config
 - Person.cs
 - Program.cs
 - Student.cs
 - StudentList.cs
- WpfApp1
 - Dependencies
 - App.xaml
 - AssemblyInfo.cs
 - MainWindow.xaml

Solution Explorer Git Changes

Properties

Name <No Name>

Type Button

Arrange by: Category

- Brush
- Layout
- Text
- Appearance

Common

Command

CommandPara...

Content GET ALL

IsCancel

IsDefault

Cursor

DataContext

New

MainWindow.xaml.cs | **MainWindow.xaml** | StudentList.cs | Student.cs | Person.cs | Program.cs

Design | XAML

31.33%

Design | XAML

TextBox (TextBox1)

```
5  xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
6  xmlns:local="clr-namespace:WpfApp1"
7  mc:Ignorable="d"
8  Title="MainWindow" Height="450" Width="800"
9  <Grid>
10 <TextBox x:Name="TextBox1" HorizontalAlignment="Left" Margin="189,43,0,0" TextWrapping="Wrap" VerticalAlignment="Top" Width="362" Height="232"/>
11 <Button Content="GET ALL" HorizontalAlignment="Left" Margin="311,328,0,0" VerticalAlignment="Top" Height="45" Width="73" RenderTransformOrigin="1.14,0.449" Click="Button_Click" />
12 </Grid>
13 </Window>
14
15
```

00% No issues found

Ln: 10 Ch: 17 SPC CRLF

Output

Show output from: Debug

WpfApp1.exe (CoreCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\6.0.16\System.Runtime.Serialization.Primitives.dll'. Skipped loading symbols. Module is optimized and the debugger option 'Just My Code' is enabled.

'WpfApp1.exe' (CoreCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\6.0.16\System.Resources.ResourceManager.dll'. Skipped loading symbols. Module is optimized and the debugger option 'Just My Code' is enabled.

'WpfApp1.exe' (CoreCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\6.0.16\System.Reflection.Emit.ILGeneration.dll'. Skipped loading symbols. Module is optimized and the debugger option 'Just My Code' is enabled.

'WpfApp1.exe' (CoreCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\6.0.16\System.Reflection.Emit.Lightweight.dll'. Skipped loading symbols. Module is optimized and the debugger option 'Just My Code' is enabled.

'WpfApp1.exe' (CoreCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\6.0.16\System.Reflection.Primitives.dll'. Skipped loading symbols. Module is optimized and the debugger option 'Just My Code' is enabled.

'WpfApp1.exe' (CoreCLR: clrhost): Loaded 'C:\Program Files\dotnet\shared\Microsoft.NETCore.App\6.0.16\System.Diagnostics.StackTrace.dll'. Skipped loading symbols. Module is optimized and the debugger option 'Just My Code' is enabled.

The program '[25020] WpfApp1.exe' has exited with code 0 (0x0).

Solution Explorer

Search Solution Explorer (Ctrl+;)

Solution 'ConsoleApp1' (2 of 2 projects)

- ConsoleApp1
 - Properties
 - References
 - App.config
 - Person.cs
 - Program.cs
 - Student.cs
 - StudentList.cs
- WpfApp1
 - Dependencies
 - App.xaml
 - AssemblyInfo.cs
 - MainWindow.xaml

Solution Explorer | Git Changes

Properties

Name: TextBox1

Type: TextBox

Arrange by: Category

- Brush
- Layout
- Text
- Appearance

Common

SelectionOpacity: 0.4

Text

UndoLimit: 100

Cursor

DataContext

IsEnabled: ☒

```
using System.Windows.Shapes;
```

```
namespace WpfApp1
```

```
{
```

```
    /// <summary>
```

```
    /// Interaction logic for MainWindow.xaml
```

```
    /// </summary>
```

```
    2 references
```

```
    public partial class MainWindow : Window
```

```
    {
```

```
        0 references
```

```
        public MainWindow()
```

```
        {
```

```
            InitializeComponent();
```

```
        }
```

```
        1 reference
```

```
        private void Button_Click(object sender, RoutedEventArgs e)
```

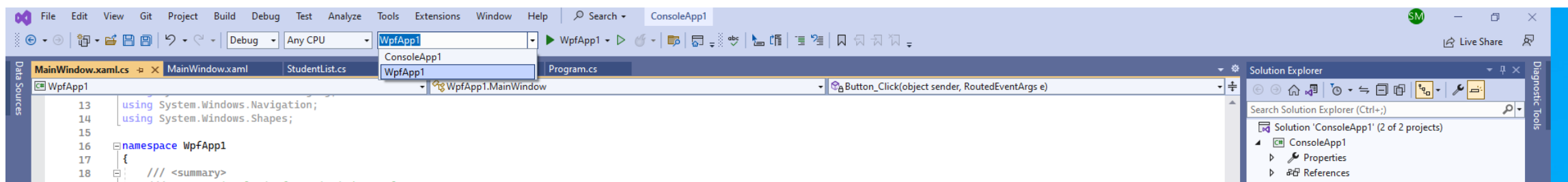
```
        {
```

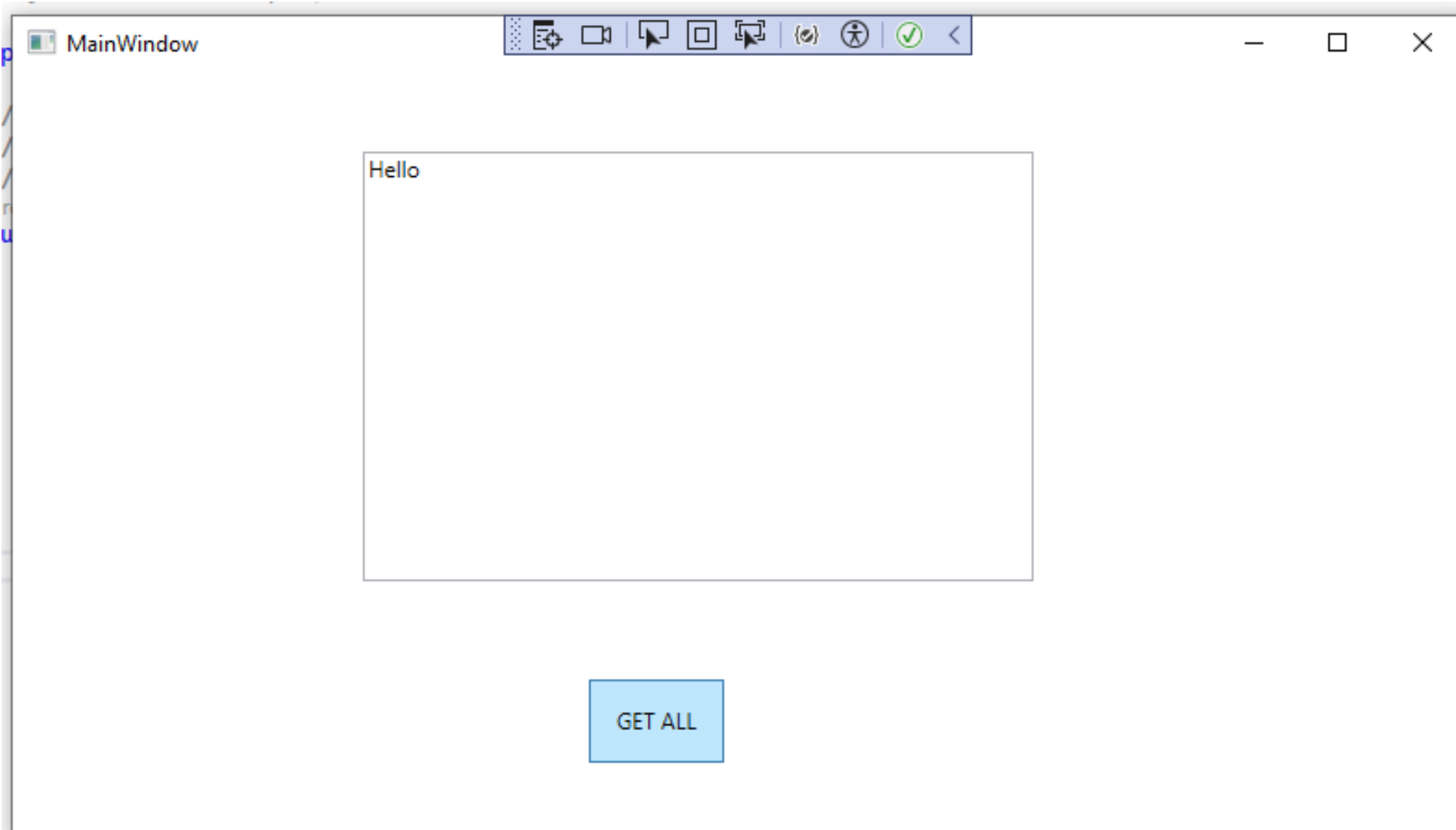
```
            TextBox1.Text = "Hello";
```

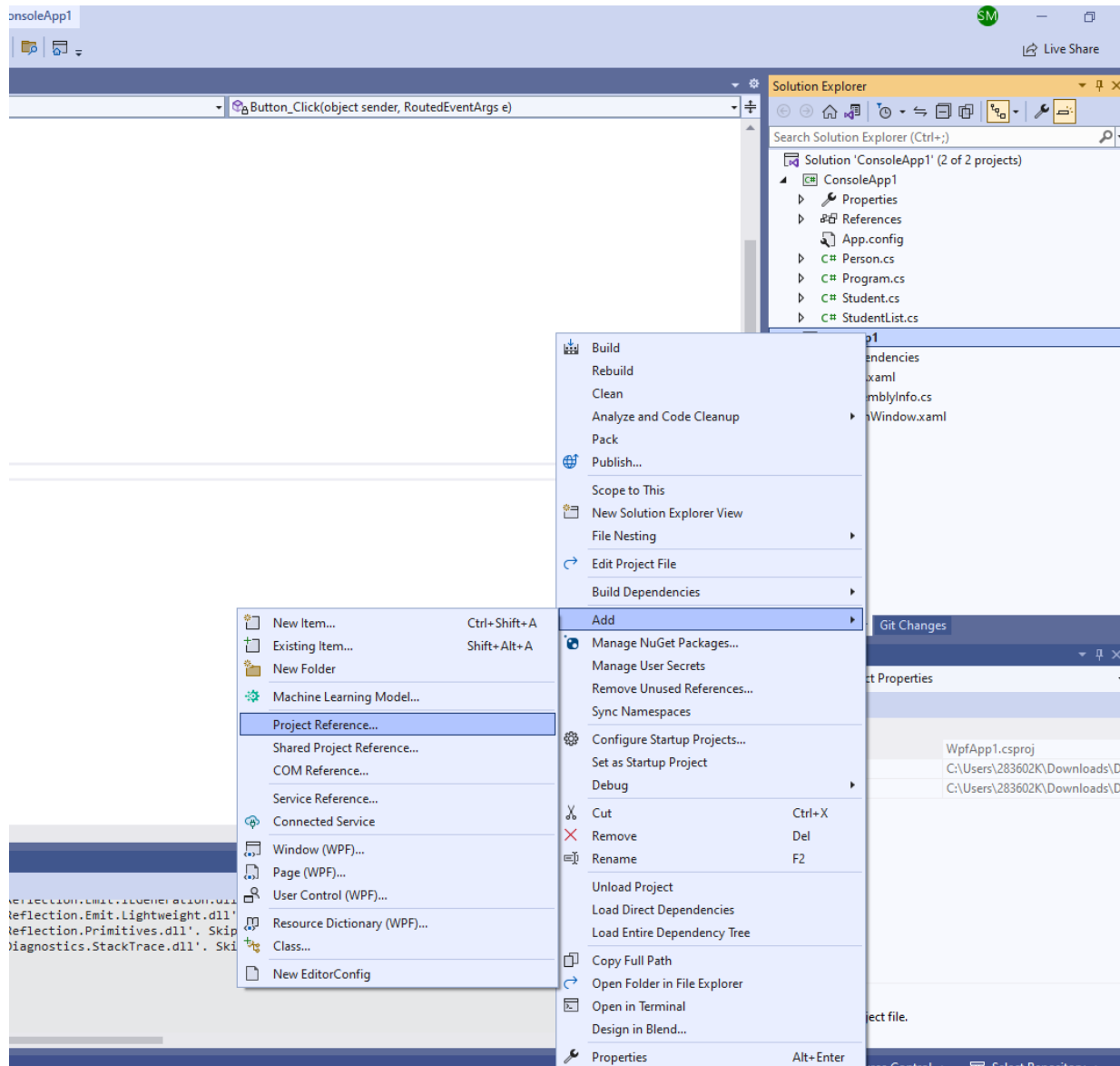
```
        }
```

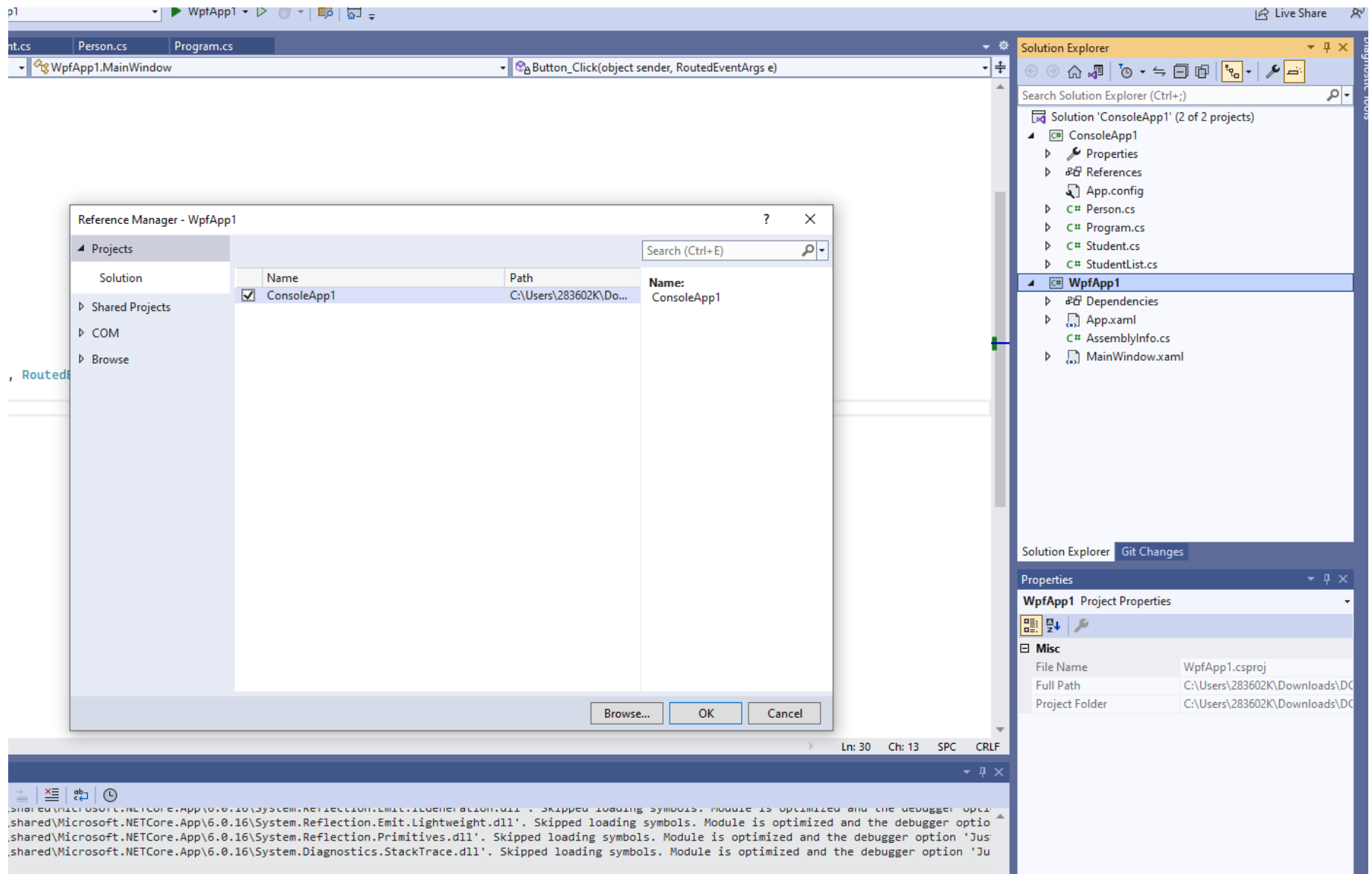
```
    }
```

```
}
```









```

using ConsoleApp1;

namespace WpfApp1
{
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    2 references
    public partial class MainWindow : Window
    {
        0 references
        public MainWindow()
        {
            InitializeComponent();
        }

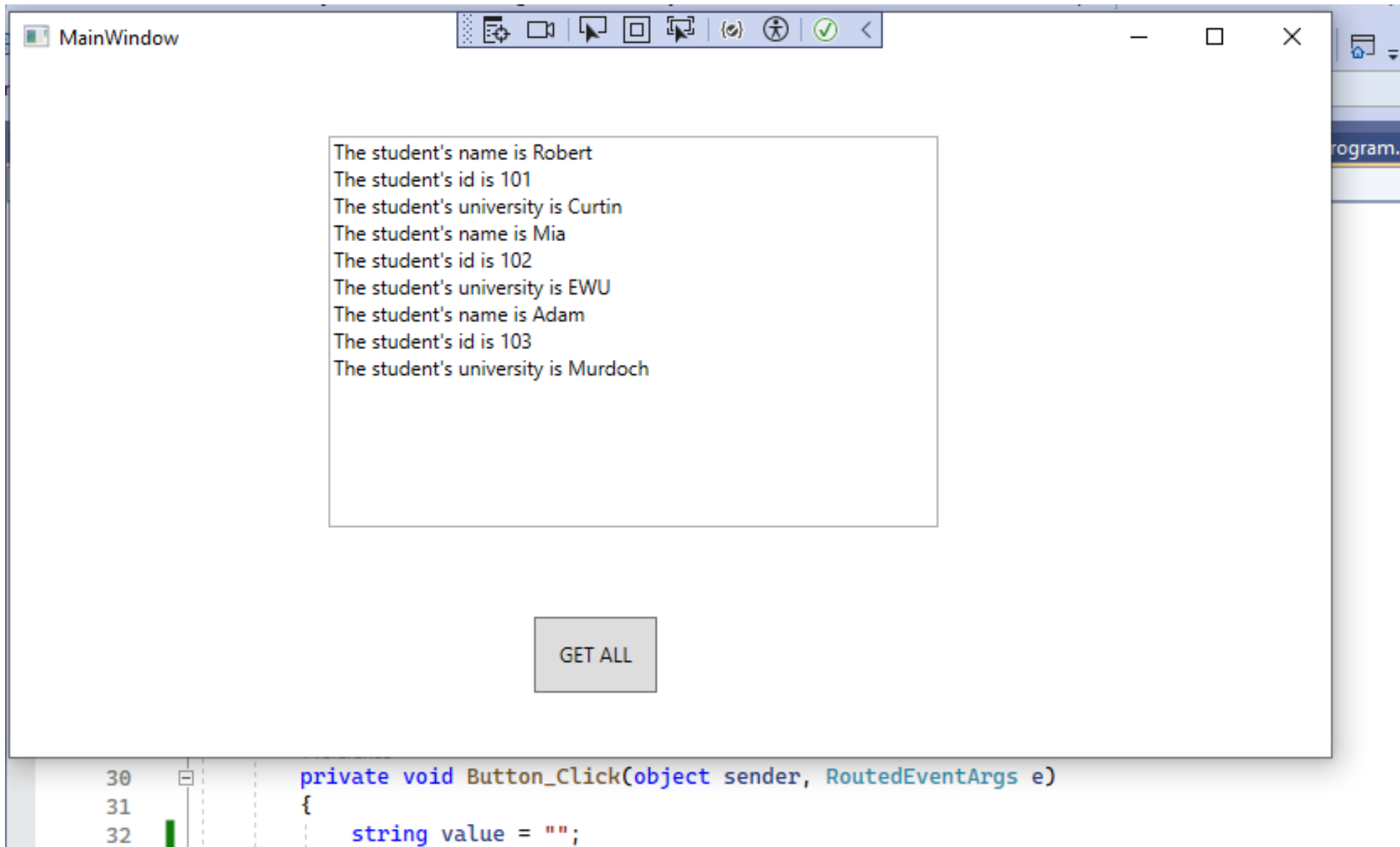
        1 reference
        private void Button_Click(object sender, RoutedEventArgs e)
        {
            string value = "";

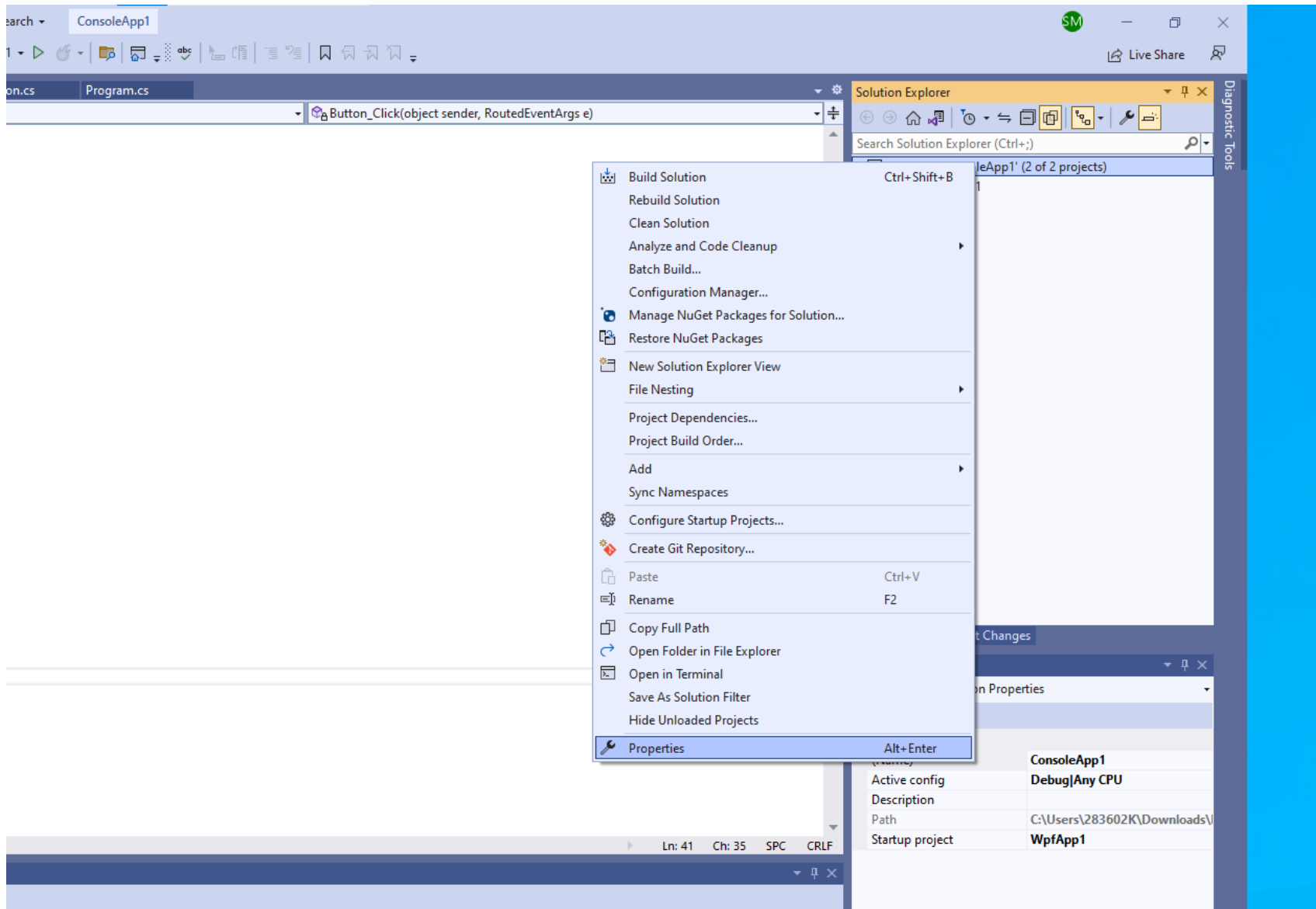
            List<Student> studentlist = StudentList.Students();

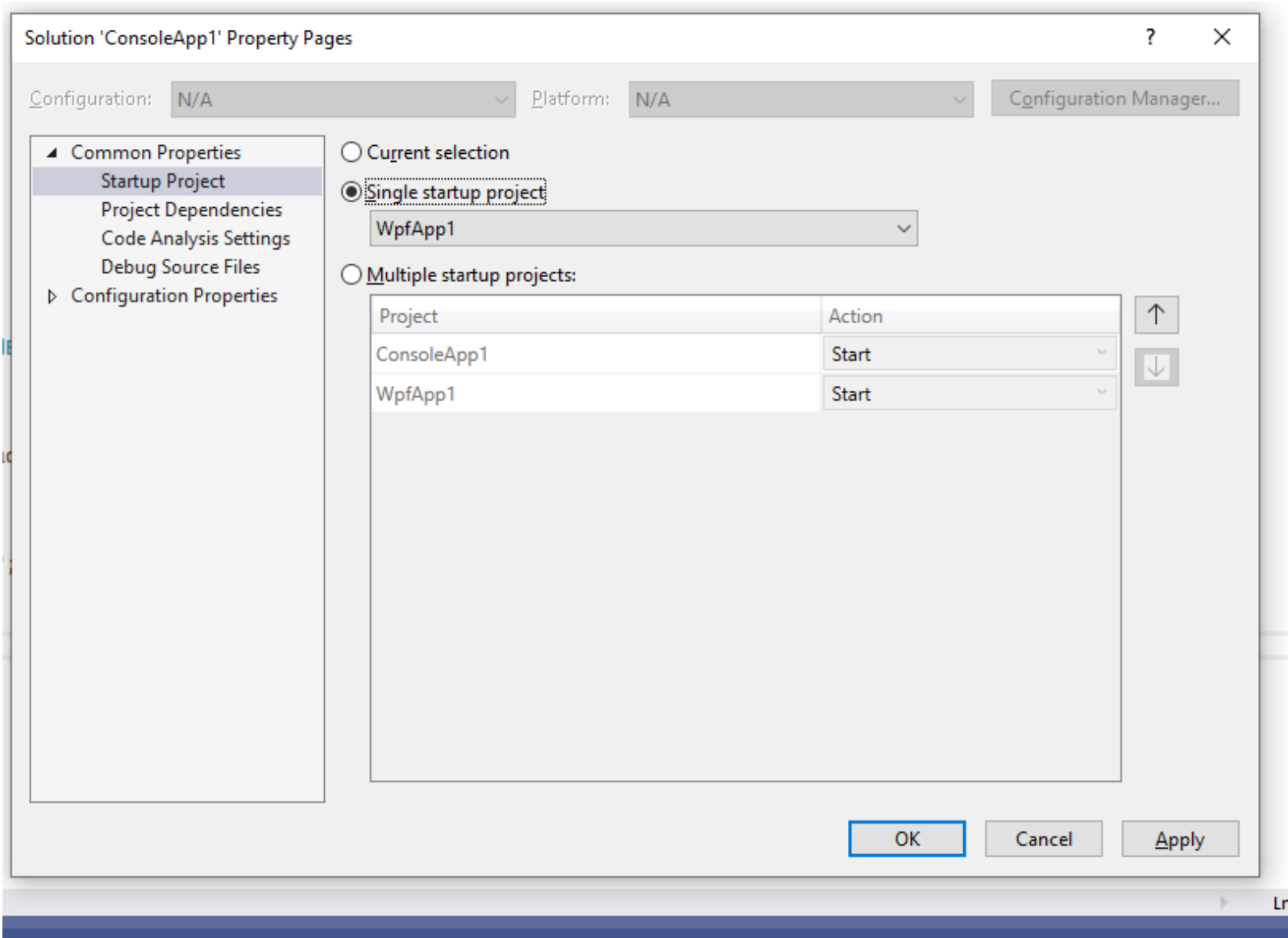
            foreach (Student student in studentlist)
            {
                value = value + student.ToString()+"\n";
            }

            TextBox1.Text = value;
        }
    }
}

```







Visual Studio IDE showing a WPF application (ConsoleApp1) running in debug mode. The application is displaying a list of student records in a console window and a dialog box.

Console Output:

```
*****
The student's name is Robert
The student's id is 101
The student's university is Curtin
*****
The student's name is Mia
The student's id is 102
The student's university is EWU
*****
The student's name is Adam
The student's id is 103
The student's university is Murdoch
*****
```

Dialog Box Content:

```
The student's name is Robert
The student's id is 101
The student's university is Curtin
The student's name is Mia
The student's id is 102
The student's university is EWU
The student's name is Adam
The student's id is 103
The student's university is Murdoch
```

GET ALL

The IDE interface includes the Solution Explorer, Diagnostic Tools (showing a 31-second session), and the Call Stack window.

Next Week

- Components
- .NET Remoting
- Java RMI
- Service Oriented Architectures

